智能排课系统
软件工程可行性报告

姚涵芳，蒋雯丽，廖语璇

第一组

2025 年 09 月 25 日

目录

1 引言 2
1.1 编写目的 2
1.2 背景 2
1.3 定义 2
1.4 参考资料 3
2 可行性研究的前提 3
2.1 要求 3
2.2 目标 4
2.3 条件、假定和限制 4
2.4 进行可行性研究的方法 4
2.5 评价尺度 4
3 对现有系统的分析 5
4 所建议的系统 5
4.1 对所建议系统的说明 5
4.2 处理流程和数据流程 6
4.3 改进之处 6
4.4 影响 6
4.5 局限性 6
4.6 技术条件方面的可行性 7
5 可选择的其他系统方案 7
6 投资及效益分析 8
6.1 支出 8
6.2 收益 8
6.3 收益投资比 9
6.4 投资回收周期 9
7 社会因素方面的可行性 9
7.1 法律方面的可行性 9
7.2 使用方面的可行性 9
8 结论 10
9 附录 10
9.1 系统总体流程图 10
9.2 系统功能模块图 11
9.3 核心业务数据流图 12
9.4 排课算法对比 12

1 引言

1.1 编写目的

本文档旨在对智能排课系统项目的可行性进行全面分析与评估，明确项目在技术、经济及社会等方面的可行性，为项目立项决策提供依据。本文档主要读者为项目评审专家、管理人员及开发团队。

1.2 背景

a 待开发软件系统名称：智能排课系统

b 项目任务提出者：庞雄文老师

项目开发者：三人开发小组，组长姚涵芳，组员廖语璇，蒋雯丽

系统目标用户：高校及培训机构教务管理人员

c 本项目源于高校教务管理工作的实际需求。传统人工排课方式效率低下且易出错，需通过信息化与智能化手段解决该问题。

1.3 定义

GB/T 8567-2016 计算机软件文档编制规范

B/S 架构 浏览器与服务器架构

API 应用程序编程接口

ORM 对象关系映射

CSP 约束满足问题

GA 遗传算法

1.4 参考资料

1 GB/T 8567-2016 计算机软件文档编制规范

2 软件工程导论 张海藩 牟永敏著

3 Vue.js 及 Django 官方开发文档

4 参考软件系统：GNU FET 正方教务管理系统排课模块

2 可行性研究的前提

2.1 要求

功能：需实现基础数据管理、自动智能排课、手动拖拽调整、冲突实时检测、课表查询与导出等功能。

性能：系统响应时间应低于 2 秒，自动排课算法需在合理时间内生成可用解。

输出：支持 Web 页面显示及 PDF 格式课表导出。

输入：通过 Web 界面输入教师、课程、教室等基础数据及排课约束规则。

安全与保密：提供用户登录认证与角色权限控制。

完成期限：本课程学期结束前。

2.2 目标

开发一个稳定易用的智能排课系统，将教务人员从手工劳动中解放出来，减少排课冲突，提高教学资源利用率，实现排课工作的自动化与智能化。

2.3 条件、假定和限制

条件：项目组成员需在开发周期内掌握必要技术栈；开发所需硬件及开源软件均已具备。

假定：项目开发过程中能获得指导老师的必要支持；用户能提供清晰的业务规则描述。

限制：项目周期固定约 16 周；开发团队仅 3 人，为学生身份，技术经验与时间均有限。

2.4 进行可行性研究的方法

本研究通过技术调研、市场分析、经济效益估算及法律法规评估等方法进行。

2.5 评价尺度

项目成功的评价尺度包括：功能满足需求、系统稳定可靠、用户界面

友好、项目按期完成、文档齐全规范。

3 对现有系统的分析

现有系统主要指人工 Excel 排课模式或集成于大型教务系统中体验不佳的排课模块。

工作负荷：教务人员需花费数天时间反复沟通核对，工作负荷极重。

费用开支：主要为人力成本，虽无直接软件费用，但时间成本高昂。

人员：需要经验丰富的教务人员操作。

设备：个人电脑。

局限性：效率低下、极易出错、难以处理复杂约束、资源利用率低、无法快速应对变更。

4 所建议的系统

4.1 对所建议系统的说明

本系统为基于 B/S 架构的 Web 应用。用户通过浏览器访问，前端负责渲染交互界面，后端提供 API 并执行核心排课算法，数据存储于 MySQL 数据库中。系统将有效克服现有系统的局限性。

4.2 处理流程和数据流程

处理流程：用户登录，管理基础数据，设置排课约束，执行自动或手动排课，验证与调整，发布课表。

数据流程：前端 UI 收集用户输入，通过 API 发送至后端，后端处理业务逻辑并操作数据库，将结果返回前端，前端展示给用户。

4.3 改进之处

将排课时间从天级缩短至分钟级。通过算法自动避免冲突，准确率近百分百。提供直观拖拽操作，调整灵活。提供多维度视图，方便查询。

4.4 影响

对设备的影响：需部署至服务器，可通过学校现有服务器或低成本云服务器实现。

对用户的影响：用户需接受简单使用培训，但因是 Web 应用，学习成本极低。

对开发的影响：开发过程需遵循软件工程规范，产出全套文档。

4.5 局限性

由于开发周期与算法复杂性，首次自动排课结果可能非全局最优解，

需辅以手动调整。系统性能可能受服务器硬件条件限制。

4.6 技术条件方面的可行性

技术方案：

前端：Vue.js 加 ElementUI

后端：Python Django 及 Django REST framework

数据库：MySQL

算法：遗传算法或约束满足问题求解

技术风险评估与对策：

风险：核心算法复杂度高。

对策：采用分阶段策略，优先实现冲突检测与手动排课，再实现简化版自动排课算法，确保核心流程可用。

结论：所选技术栈成熟开源社区活跃，团队成员具备学习能力，技术可行性高。

5 可选择的其他系统方案

方案一被选方案：Vue.js 加 Django 全栈方案。优点技术组合合理开发效率高资料丰富。

方案二备选方案：Reactｔ加 Spring Boot 方案。优点 Java 技术体系稳健。缺点团队对 Spring Boot 熟悉度可能不如 Django，开发效率稍低。

经评估，选择方案一更符合本项目周期短重效率的特点。

6 投资及效益分析

6.1 支出

一次性支出：

人力成本：3 人 4 月, 学生人力成本估算约 0 元。

软硬件成本：使用现有开发设备及开源软件成本 0 元。部署可选用学生优惠云服务器约 100 元每年。

非一次性支出：无。

6.2 收益

一次性收益：无直接经济收益。

非一次性收益：

节省的人力成本：每年为教务部门节省大量排课时间。

提高资源利用率：优化教室教师资源分配，间接产生经济效益。

减少教学事故：避免冲突带来的潜在损失。

不可定量的收益：提升教务管理信息化水平和满意度。

6.3 收益投资比

由于投资极低，而收益尤其是间接收益和不可量化收益显著，收益投资比非常高。

6.4 投资回收周期

项目投入几乎为零，因此投资回收周期极短，在投入运行后即可视为产生回报。

7 社会因素方面的可行性

7.1 法律方面的可行性

本项目所有技术组件 Vue.js Django MySQL 等均采用开源协议 MIT BSD GPL 等，允许免费用于学习开发和商业用途，在法律上完全可行。项目为自主开发，无侵权问题。

7.2 使用方面的可行性

系统面向教务管理员，提供完善的 Web 界面和操作指引，符合用户现有操作习惯，使用可行性高。

8 结论

综上所述：

技术可行性：成立。技术栈成熟，团队技术能力可及，虽有算法挑战但有稳妥对策。

经济可行性：成立。项目投入极低，间接收益和战略收益显著。

社会可行性：成立。法律上无侵权问题，操作上易于使用。

结论意见：本项目值得立即开始进行。

9 附录

9.1 系统总体流程图

9.2 系统功能模块图

智能排课系统功能模块结构

用户管理模块

- 用户登录/注销
- 角色权限管理
- 个人信息维护

基础数据管理模块

- 教师信息管理
- 课程信息管理
- 教室信息管理
- 班级信息管理

约束规则管理模块

- 时间约束设置
- 场地约束设置
- 教师约束设置
- 课程关联约束

智能排课模块

- 自动排课算法
- 手动排课调整
- 冲突实时检测
- 排课结果优化

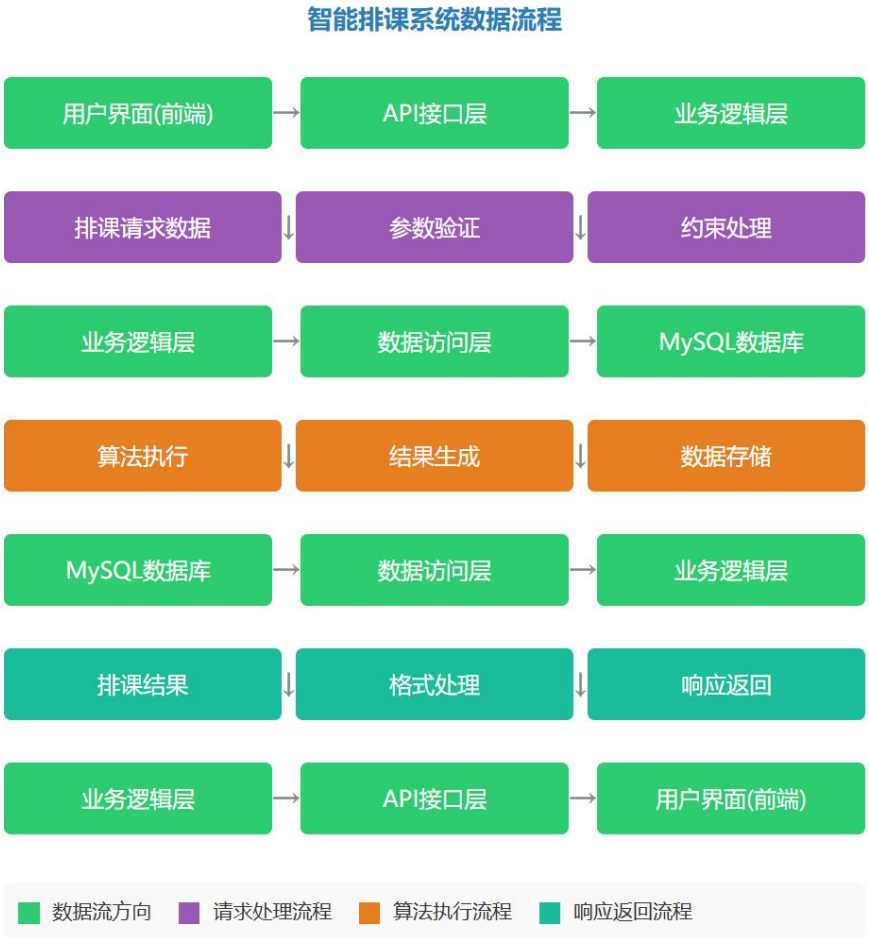
课表管理模块

- 课表查询浏览
- 课表导出(PDF/Excel)
- 课表发布管理
- 课表版本控制

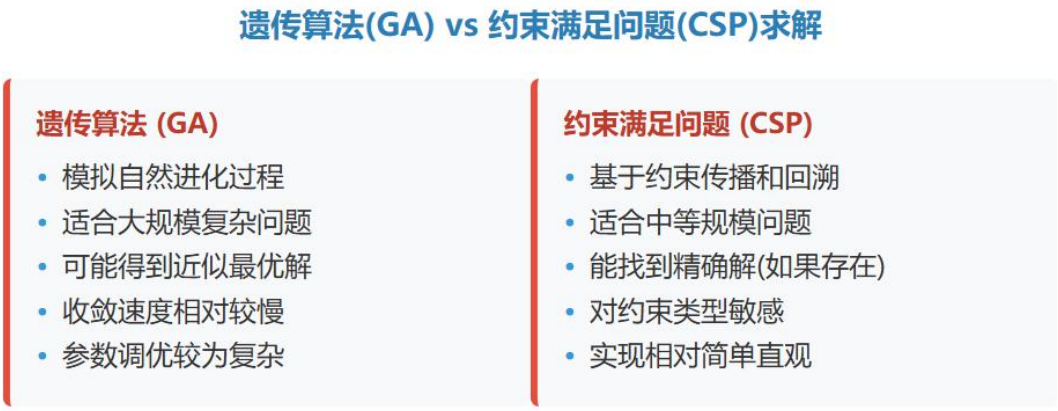
系统设置模块

- 学期时间设置
- 系统参数配置
- 数据备份恢复
- 操作日志查看

9.3 核心业务数据流图



9.4 排课算法对比



9.5 技术栈列表及版本

层级	技术组件	推荐版本	说明
前端	Vue.js	3.x	渐进式 JavaScript 框架，易于学习和集成，生态丰富。
前端	Element-Plus	2.x	基于 Vue 3 的桌面端 UI 组件库，提供丰富、美观的组件。
前端	Axios	1.x	基于 Promise 的 HTTP 客户端,用于向后端发送 API 请求。
前端	ECharts	5.x	强大的数据可视化库，用于课表图形化展示。
后端	Python	3.8+	解释型编程语言，语法简洁，在算法和 Web 开发中应用广泛。
后端	Django	4.x	高级 Python Web 框架，以其"开箱即用"和稳健著称。
后端	Django REST Framework (DRF)	3.14	用于构建 Web API 的强大且灵活的工具包，与 Django 无缝集成。
后端	Django CORS Headers	4.x	用于处理前后端分离项目中的跨域请求。
数据库	MySQL	8.0	最流行的开源关系型数据库，性能稳定，社区活跃。

层级	技术组件	推荐版本	说明
数据库	MySQL Client / ORM	-	使用 Django 自带的 ORM 进行数据库操作，无需直接编写 SQL。
开发与部署	Git	2.x	分布式版本控制系统，用于代码管理和团队协作。
开发与部署	Node.js	18.x	JavaScript 运行时，用于构建和运行前端工具。
开发与部署	Vite	4.x	下一代前端构建工具，提供极速的开发服务器启动。
开发与部署	Docker	20.x	容器化平台，用于实现环境一致性和简化部署。
开发与部署	Nginx	1.x	高性能的 HTTP 和反向代理服务器，用于部署前端和代理后端 API。